

Artificial Intelligence Reinforcement Learning

Session 2
Search and Decision Making

Ali Ismail-Fawaz

ENSISA, Université Haute-Alsace

Contents

Problem Solving Through Search

Search Algorithms

Heuristic Search

Adversarial Search

The Search Problem

How can an intelligent agent choose
the best sequence of actions?

Examples

- ▶ Finding the shortest route on Google Maps
- ▶ Solving a maze
- ▶ Planning a robot's movements
- ▶ Choosing the next move in a chess game

Today's Question

How can an AI system efficiently search for the best solution among millions of possibilities?

Search as a Problem

Definition

A search problem consists of finding a sequence of actions that transforms an **initial state** into a **goal state**.



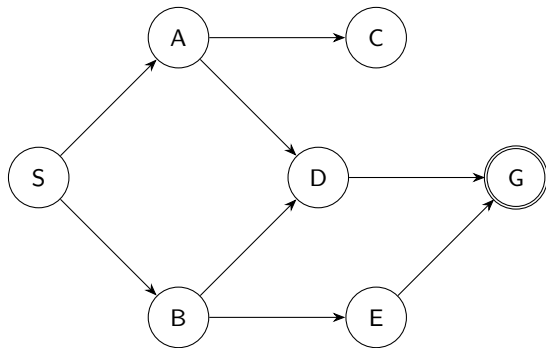
Examples

- ▶ A GPS searches for a route from your location to your destination.
- ▶ A robot searches for a collision-free path.
- ▶ A chess engine searches for a sequence of winning moves.

State Space Representation

State Space

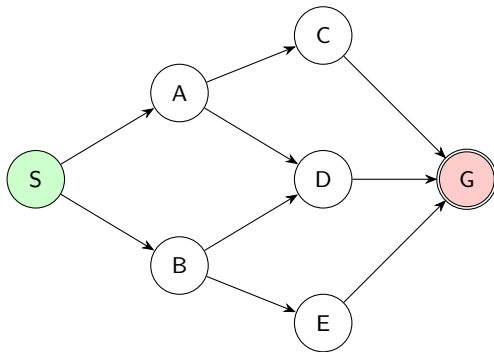
A **state space** is the collection of all possible states that an agent can reach by performing actions.



- ▶ Nodes represent **states**.
- ▶ Edges represent **actions**.
- ▶ The objective is to find a path from the initial state S to the goal state G .

How Should We Explore the State Space?

Given a state space, how should an intelligent agent find the goal?



Challenges

- ▶ There may be many possible paths.
- ▶ Some paths are longer than others.
- ▶ Exploring every possibility may be too expensive.

Contents

Problem Solving Through Search

Search Algorithms

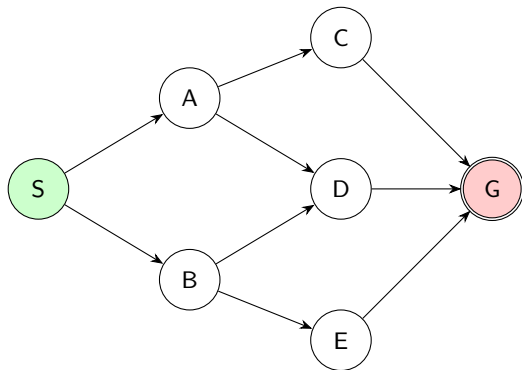
Heuristic Search

Adversarial Search

Exploring the State Space

Key Observation

A search algorithm repeatedly chooses one state to expand until it reaches the goal.



The Central Question

Which state should be explored next?

Uninformed Search

Idea

Suppose the agent has **no additional knowledge** about which path is better. The only information available is the structure of the state space.

The simplest strategies are:

- ▶ Breadth-First Search (BFS)
- ▶ Depth-First Search (DFS)

Question

Should we explore states **level by level**, or should we go **as deep as possible** first?

Uninformed Search

Idea

Suppose the agent has **no additional knowledge** about which path is better. The only information available is the structure of the state space.

The simplest strategies are:

- ▶ Breadth-First Search (BFS)
- ▶ Depth-First Search (DFS)

Question

Should we explore states **level by level**, or should we go **as deep as possible** first?

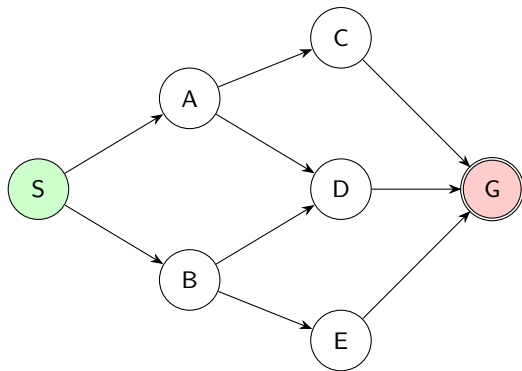
Algorithms vs. Artificial Intelligence

Algorithms Course	Artificial Intelligence
Graph traversal	State-space search
Visit graph vertices	Explore possible states
Graph is the input	Problem is modeled as a graph
Goal: traversal/connectivity	Goal: find a solution (or the best one)

Breadth-First Search (BFS)

Main Idea

Breadth-First Search explores the state space **level by level**. It expands all states at the current depth before moving to the next depth.



Expansion Order: $S \rightarrow A, B \rightarrow C, D, E \rightarrow G$

Breadth-First Search: Properties

Characteristics

- ▶ Explores the state space level by level.
- ▶ Uses a **First-In, First-Out (FIFO)** queue.
- ▶ Complete: finds a solution if one exists (finite search space).
- ▶ Optimal when all actions have the same cost.

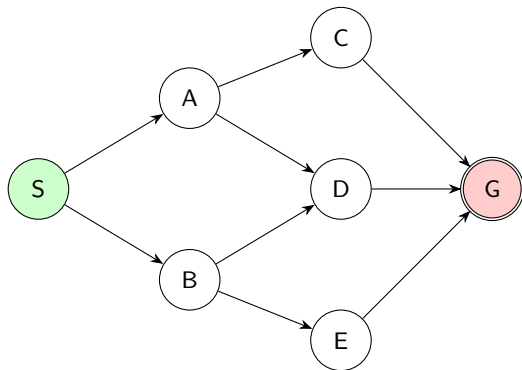
Limitations

- ▶ Must store all frontier nodes in memory.
- ▶ Memory requirements grow rapidly as the state space becomes larger.

Depth-First Search (DFS)

Main Idea

Depth-First Search (DFS) explores one path as deeply as possible before backtracking to explore alternative paths.



Possible Expansion Order: $S \rightarrow A \rightarrow C \rightarrow G$

Depth-First Search: Properties

Characteristics

- ▶ Explores one branch at a time before backtracking.
- ▶ Uses a **Last-In, First-Out (LIFO)** stack (or recursion).
- ▶ Requires much less memory than BFS.

Limitations

- ▶ Not guaranteed to find the shortest solution.
- ▶ May spend a long time exploring an unpromising branch.
- ▶ May fail to terminate in infinite search spaces.

Contents

Problem Solving Through Search

Search Algorithms

Heuristic Search

Adversarial Search

Beyond Blind Search

Observation

Breadth-First Search and Depth-First Search explore the state space without any knowledge about which path is more promising.

Can we guide the search
using knowledge about the problem?

Idea

Instead of exploring blindly, estimate which states are **closer to the goal** and explore them first.

Heuristics

What is a Heuristic?

A **heuristic** is a rule or estimate that helps an intelligent agent decide which state is more promising to explore next.

Examples

- ▶ GPS estimates the remaining distance to the destination.
- ▶ A chess engine evaluates which board positions are favorable.
- ▶ A robot estimates how close it is to its target.

$h(n)$ = Estimated cost from state n to the goal

Key Idea

A heuristic does **not guarantee** the correct answer, but it often helps the agent find a solution much faster.

A* Search

Main Idea

A* chooses the next state by considering:

- ▶ the cost already incurred, and
- ▶ the estimated cost remaining to reach the goal.

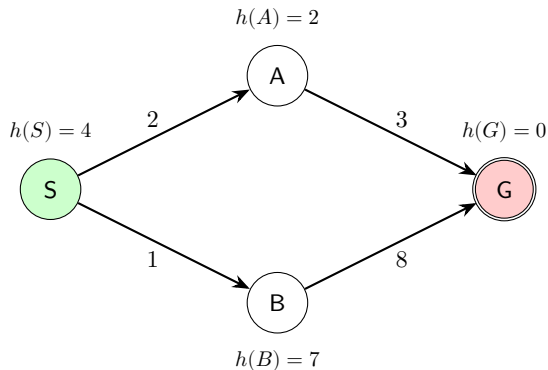
$$f(n) = g(n) + h(n)$$

- ▶ $g(n)$: cost from the start state to n
- ▶ $h(n)$: estimated cost from n to the goal
- ▶ $f(n)$: estimated total cost of a solution through n

Key Idea

Rather than exploring every possibility, A* prioritizes states that appear to lead to the lowest-cost solution.

Example: A* Search



$$f(n) = g(n) + h(n)$$

State	$g(n)$	$h(n)$	$f(n)$
A	2	2	4
B	1	7	8

Example Calculation

For state A:

$$g(A) = 2$$

because the path from S to A costs 2.

$$h(A) = 2$$

is the heuristic estimate from A to G .
Therefore,

$$f(A) = 2 + 2 = 4.$$

A* Decision: A* expands A first because

$$f(A) = 4 < f(B) = 8.$$

Where do the values come from?

- ▶ Numbers on the arrows are the **actual action costs**.
- ▶ $g(n)$ is the total cost accumulated from S to state n .
- ▶ $h(n)$ is an estimate of the remaining cost to G , computed using a problem-specific heuristic.

A* vs. Dijkstra's Algorithm

Feature	Dijkstra	A*
Uses path cost $g(n)$	✓	✓
Uses heuristic $h(n)$	×	✓
Node selection	Smallest $g(n)$	Smallest $g(n) + h(n)$
Search strategy	Uniform exploration	Goal-directed exploration
Typical speed	Slower	Faster (with a good heuristic)
Guarantees optimal solution	✓	✓*

Key Insight

Dijkstra explores the graph based only on the cost accumulated so far, whereas A* also estimates the remaining cost to the goal, allowing it to focus the search toward promising states.

A is optimal when the heuristic is **admissible** (it never overestimates the true remaining cost).

Applications of A* Search

Navigation

- ▶ GPS route planning
- ▶ Maps and navigation systems

Video Games

- ▶ NPC pathfinding
- ▶ Real-time strategy games

Robotics

- ▶ Autonomous robots
- ▶ Warehouse automation

Autonomous Vehicles

- ▶ Route planning
- ▶ Obstacle avoidance

Why A*?

A* finds high-quality paths efficiently by combining the **actual cost** traveled with a **heuristic estimate** of the remaining distance.

Why is A* an AI Algorithm?

Without Intelligence

Algorithms such as BFS or Dijkstra explore the search space using only the information available in the graph.

Adding Intelligence

A* introduces a **heuristic**:

$$f(n) = g(n) + h(n)$$

The heuristic encodes **knowledge about the problem**, allowing the agent to make informed decisions instead of exploring blindly.

Example

For a GPS application, the heuristic may be the **straight-line distance** to the destination. Although this is only an estimate, it guides the search toward promising routes.

Key Takeaway

Artificial Intelligence is not only about searching, it is about **using knowledge to search intelligently**.

Contents

Problem Solving Through Search

Search Algorithms

Heuristic Search

Adversarial Search

From Path Finding to Games

Search So Far

In the problems we have studied:

- ▶ The environment was static.
- ▶ The agent searched for the best path.
- ▶ Nothing actively opposed the agent.

A New Challenge

Many AI applications involve an **opponent** making decisions.

Examples include:

- ▶ Chess
- ▶ Tic-Tac-Toe
- ▶ Checkers
- ▶ Go

Key Question

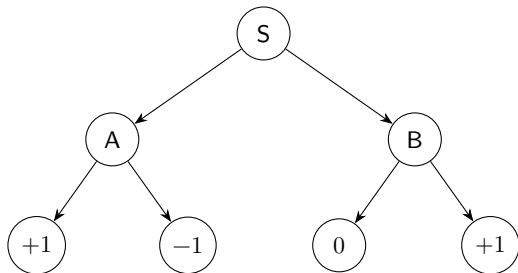
How can an AI choose the best move when another intelligent player is trying to defeat it?

Game Trees

Representing a Game

A game can be represented as a **tree** where:

- ▶ Each node represents a game state.
- ▶ Each edge represents a possible move.
- ▶ The root is the current position.
- ▶ The leaves are possible game outcomes.



Terminal States

- ▶ +1: AI wins
- ▶ 0: Draw
- ▶ -1: AI loses

Key Idea

Searching in games means exploring the game tree to determine which move leads to the best possible outcome.

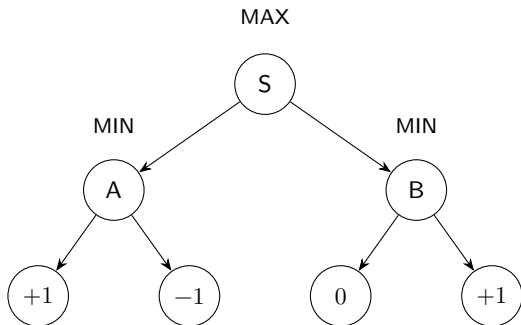
Two Players: MAX and MIN

MAX

- ▶ Represents the AI.
- ▶ Tries to obtain the highest value.
- ▶ Chooses the move that maximizes the outcome.

MIN

- ▶ Represents the opponent.
- ▶ Tries to obtain the lowest value for the AI.
- ▶ Chooses the move that minimizes the outcome.



How Are the Values Chosen?

- ▶ At node **A**, MIN chooses: $\min(+1, -1) = -1$
- ▶ At node **B**, MIN chooses: $\min(0, +1) = 0$
- ▶ At node **S**, MAX chooses: $\max(-1, 0) = 0$

Therefore, the AI chooses move **B**.

Key Assumption

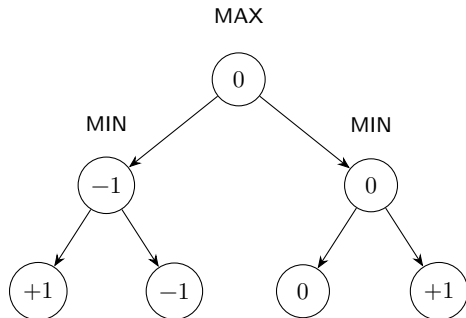
The AI assumes that the opponent will always choose the move that is **best for them**, even if it is the **worst for us**.

The Minimax Algorithm

Idea

Minimax assumes both players play optimally:

- ▶ **MAX (AI)** chooses the move with the highest value.
- ▶ **MIN (Opponent)** chooses the move with the lowest value.



Value Propagation

1. MIN selects the lowest value in each subtree.
2. Those values are propagated to the parent.
3. MAX selects the highest propagated value.

$$\min(+1, -1) = -1$$

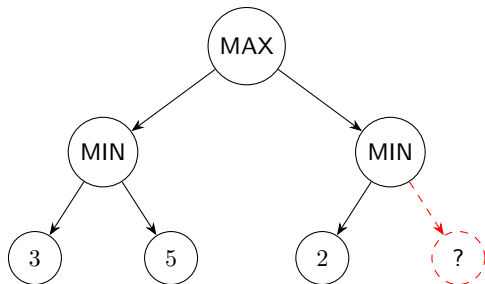
$$\min(0, +1) = 0$$

$$\boxed{\max(-1, 0) = 0}$$

Alpha-Beta Pruning

Key Idea

During the search, some branches can no longer influence the final decision. Instead of exploring them, we simply **prune** (skip) those branches.



Observation

1. Suppose MAX already has a move worth 3.
2. When exploring the second branch, MIN already found a value of 2.
3. Since MIN can only keep the value at 2 or make it even smaller, MAX will never choose this branch.
4. Therefore, the remaining node does not need to be explored.

Result

Alpha-Beta Pruning produces the **same optimal decision** as Minimax while evaluating far fewer game states.

Alpha and Beta Values

Alpha (α)

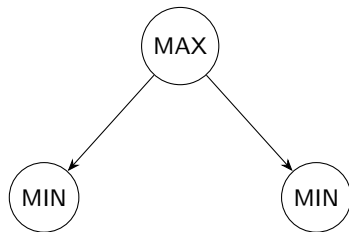
The best value that **MAX** can guarantee along the current search path. MAX tries to **increase** α

Beta (β)

The best value that **MIN** can guarantee along the current search path. MIN tries to **decrease** β

Example

Suppose MAX has already found a move with value **5** ($\alpha = 5$). While exploring another branch, MIN already found a move with value **3** ($\beta = 3$). Since MIN can only keep the value at 3 or make it even smaller, MAX will never choose this branch. Therefore, the remaining nodes are pruned.



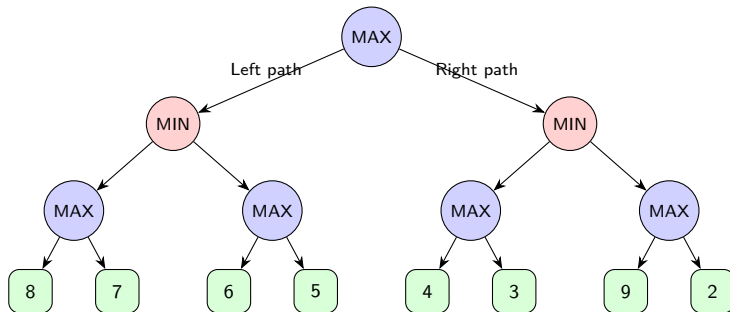
α increases

β decreases

Pruning Rule

When $\alpha \geq \beta$, the remaining branches cannot affect the final decision and are pruned.

Example Game: The Treasure Paths



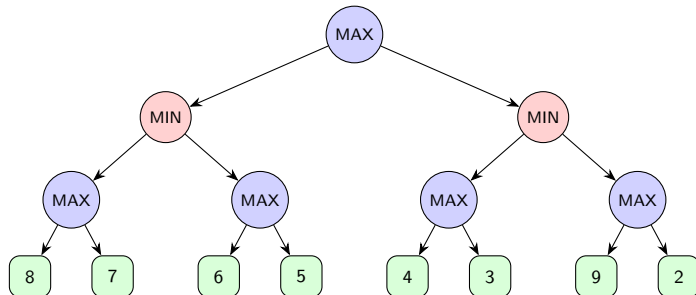
Rules

- ▶ MAX is the AI.
- ▶ MIN is the opponent.
- ▶ MAX wants the highest score.
- ▶ MIN wants the lowest score for the AI.

Question

Which first path should the AI choose?

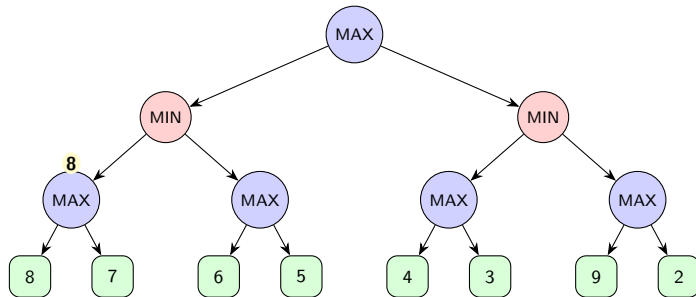
Solving the Game with Minimax



Minimax

Evaluate the complete game tree from the leaves upward.

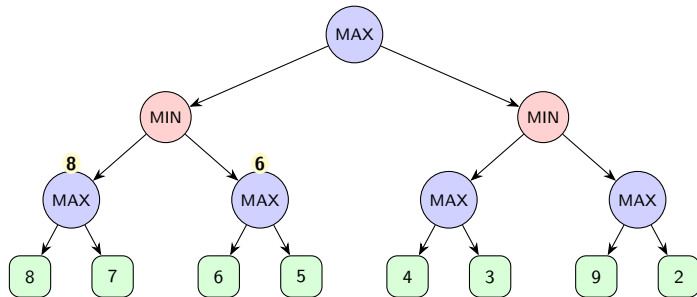
Solving the Game with Minimax



Step 1

$$\max(8, 7) = 8$$

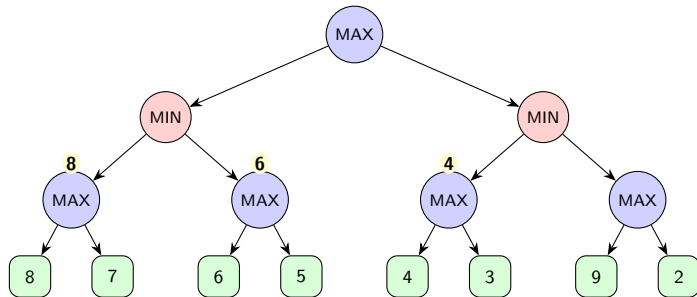
Solving the Game with Minimax



Step 2

$$\max(6, 5) = 6$$

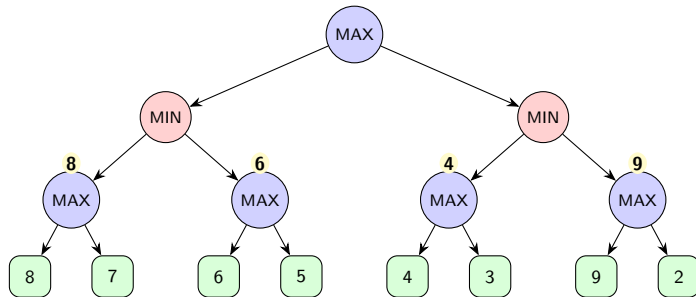
Solving the Game with Minimax



Step 3

$$\max(4, 3) = 4$$

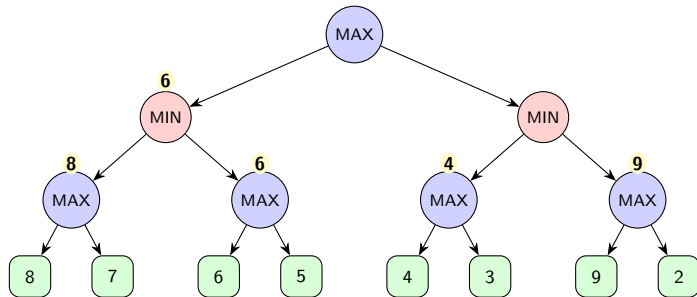
Solving the Game with Minimax



Step 4

$$\max(9, 2) = 9$$

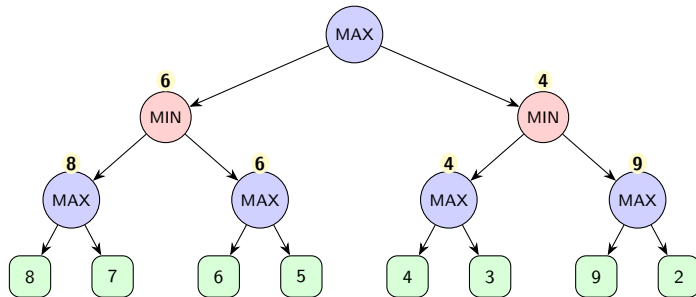
Solving the Game with Minimax



Step 5

$$\min(8, 6) = 6$$

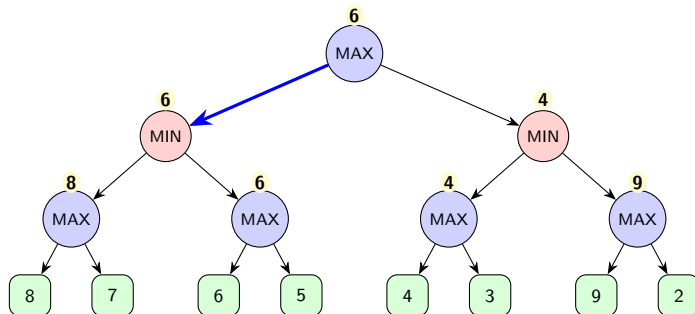
Solving the Game with Minimax



Step 6

$$\min(4, 9) = 4$$

Solving the Game with Minimax



Final Decision

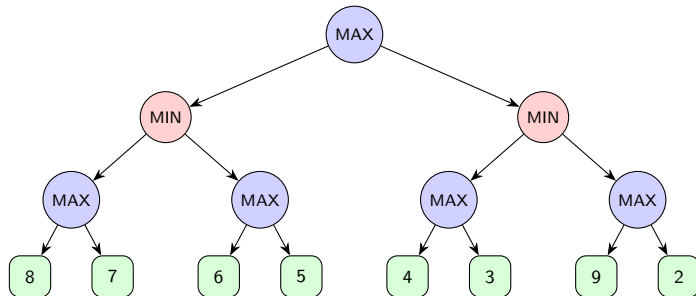
$$\max(6, 4) = 6$$

The AI chooses the left path.

Work Required

8 of 8 leaves evaluated

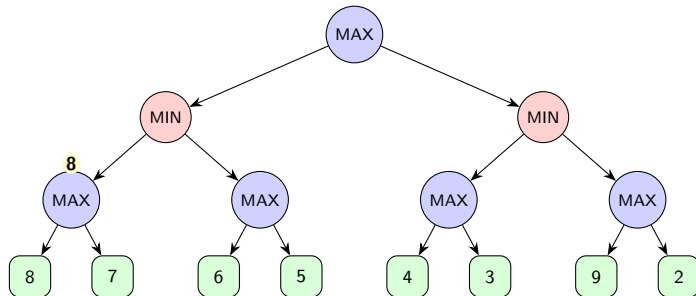
Solving the Same Game with Alpha-Beta Pruning



Search Order

Explore the tree from left to right.

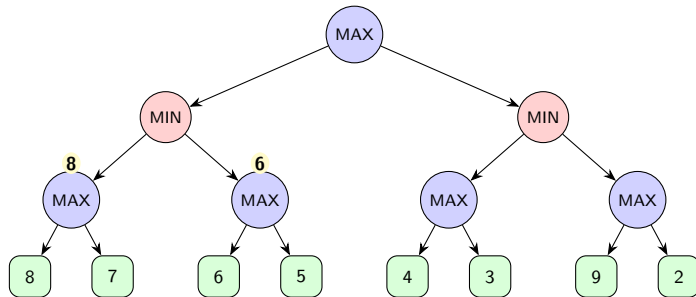
Solving the Same Game with Alpha-Beta Pruning



Left Subtree

$$\max(8, 7) = 8$$

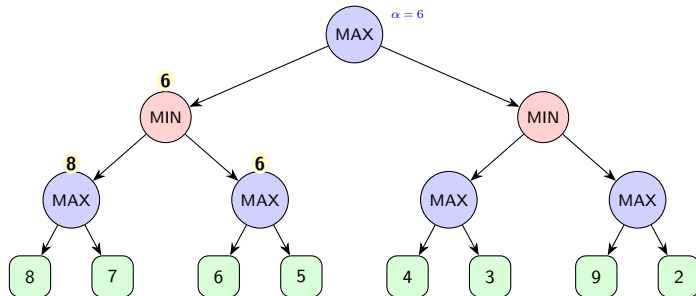
Solving the Same Game with Alpha-Beta Pruning



Left Subtree

$$\max(6, 5) = 6$$

Solving the Same Game with Alpha-Beta Pruning



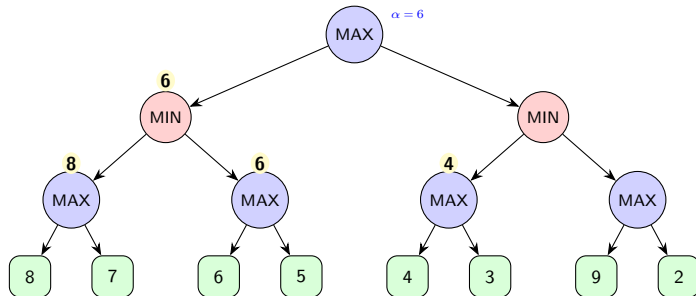
Root Update

$$\min(8, 6) = 6$$

MAX can now guarantee:

$$\alpha = 6$$

Solving the Same Game with Alpha-Beta Pruning

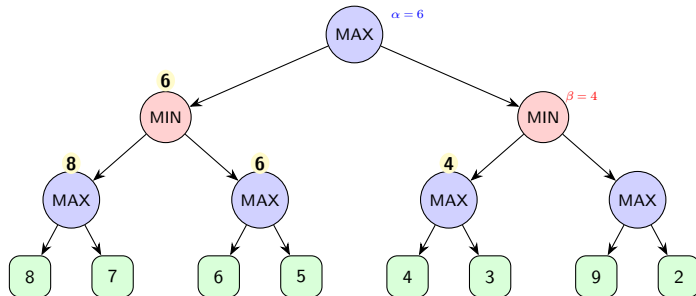


Right Subtree

The first MAX node gives:

$$\max(4, 3) = 4$$

Solving the Same Game with Alpha-Beta Pruning



Beta Update

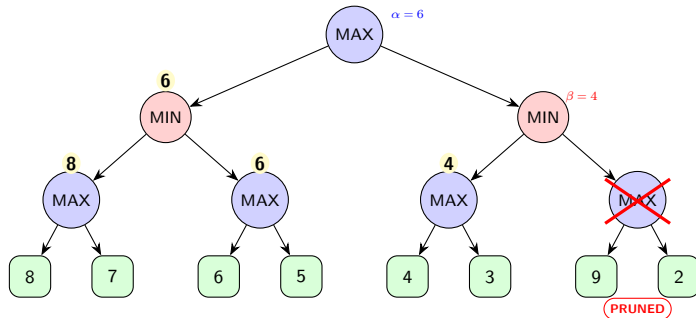
MIN can guarantee:

$$\beta = 4$$

Therefore:

$$\alpha = 6 \geq \beta = 4$$

Solving the Same Game with Alpha-Beta Pruning

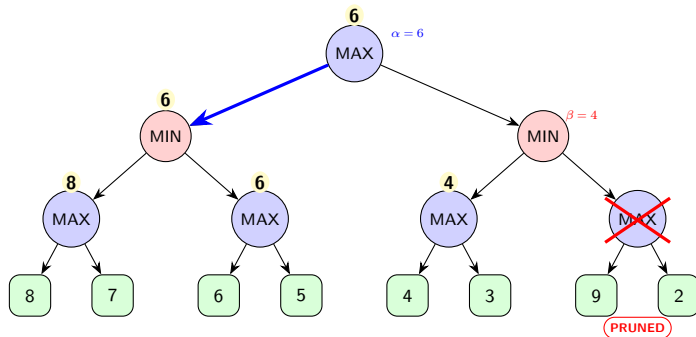


Prune

The remaining branch cannot improve MAX's decision.

The leaves 9 and 2 are not evaluated.

Solving the Same Game with Alpha-Beta Pruning



Final Decision

The AI still chooses the left path with value:

6

Work Required

6 of 8 leaves evaluated

Minimax vs. Alpha-Beta Pruning

Property	Minimax	Alpha-Beta
Finds the optimal decision	Yes	Yes
Leaf evaluations in our example	8	6
Explores every branch	Yes	Not always
Prunes irrelevant branches	No	Yes
Worst-case time complexity	$O(b^d)$	$O(b^d)$
Best-case time complexity	$O(b^d)$	$O(b^{d/2})$

Main Takeaway

Alpha-Beta produces the **same decision** as Minimax, but it can evaluate **far fewer game states**.

$$\alpha \geq \beta \implies \text{prune the remaining branch}$$

The Monte Carlo Idea

Core Principle

Monte Carlo methods estimate an unknown quantity by generating many random samples and averaging the observed outcomes.

Estimating π

Randomly sample points inside a square containing a circle.

The probability that a point falls inside the circle is

$$\frac{\text{Area of circle}}{\text{Area of square}} = \frac{\pi r^2}{(2r)^2} = \frac{\pi}{4}.$$

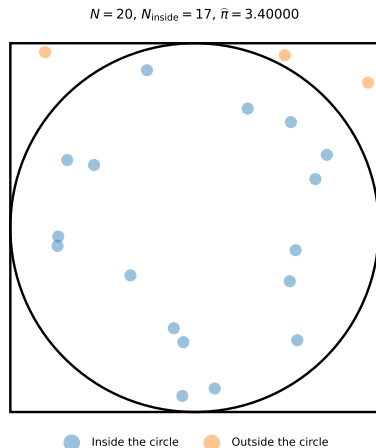
Therefore,

$$\pi \approx 4 \times \frac{N_{\text{inside}}}{N_{\text{total}}}$$

The Monte Carlo Idea

Core Principle

Monte Carlo methods estimate an unknown quantity by generating many random samples and averaging the observed outcomes.



Estimating π

Randomly sample points inside a square containing a circle.

The probability that a point falls inside the circle is

$$\frac{\text{Area of circle}}{\text{Area of square}} = \frac{\pi r^2}{(2r)^2} = \frac{\pi}{4}.$$

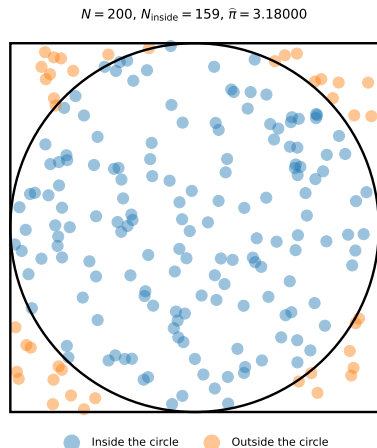
Therefore,

$$\pi \approx 4 \times \frac{N_{\text{inside}}}{N_{\text{total}}}$$

The Monte Carlo Idea

Core Principle

Monte Carlo methods estimate an unknown quantity by generating many random samples and averaging the observed outcomes.



Estimating π

Randomly sample points inside a square containing a circle.

The probability that a point falls inside the circle is

$$\frac{\text{Area of circle}}{\text{Area of square}} = \frac{\pi r^2}{(2r)^2} = \frac{\pi}{4}.$$

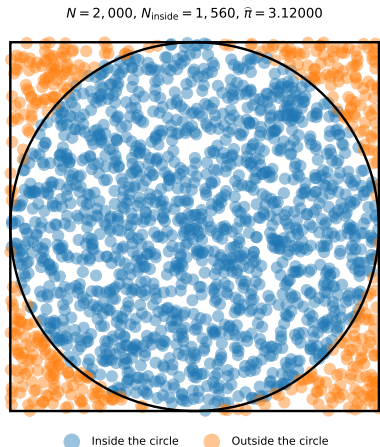
Therefore,

$$\pi \approx 4 \times \frac{N_{\text{inside}}}{N_{\text{total}}}$$

The Monte Carlo Idea

Core Principle

Monte Carlo methods estimate an unknown quantity by generating many random samples and averaging the observed outcomes.



Estimating π

Randomly sample points inside a square containing a circle.

The probability that a point falls inside the circle is

$$\frac{\text{Area of circle}}{\text{Area of square}} = \frac{\pi r^2}{(2r)^2} = \frac{\pi}{4}.$$

Therefore,

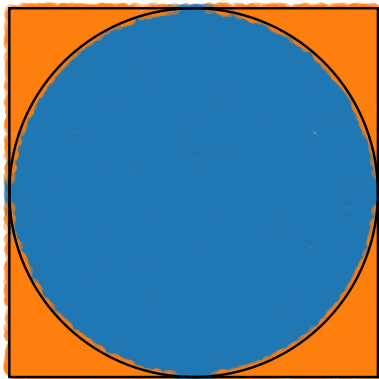
$$\pi \approx 4 \times \frac{N_{\text{inside}}}{N_{\text{total}}}$$

The Monte Carlo Idea

Core Principle

Monte Carlo methods estimate an unknown quantity by generating many random samples and averaging the observed outcomes.

$N = 20,000$, $N_{\text{inside}} = 15,652$, $\hat{\pi} = 3.13040$



● Inside the circle ● Outside the circle

Estimating π

Randomly sample points inside a square containing a circle.

The probability that a point falls inside the circle is

$$\frac{\text{Area of circle}}{\text{Area of square}} = \frac{\pi r^2}{(2r)^2} = \frac{\pi}{4}.$$

Therefore,

$$\pi \approx 4 \times \frac{N_{\text{inside}}}{N_{\text{total}}}$$

From Monte Carlo Sampling to MCTS

Same General Idea

In the previous example, random samples were used to estimate the value of π .

In Monte Carlo Tree Search, random simulations are used to estimate
the value of possible moves.

Estimating π

- ▶ Sample random points.
- ▶ Observe inside or outside.
- ▶ Average the outcomes.

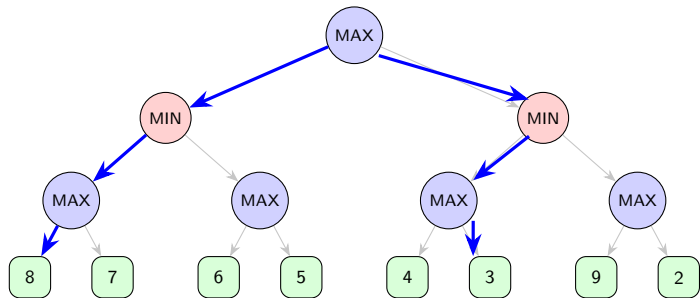
Monte Carlo Tree Search

- ▶ Sample possible game continuations.
- ▶ Observe wins, losses, or rewards.
- ▶ Average the outcomes.

Key Connection

Monte Carlo methods replace exact computation with repeated sampling and statistical estimation.

Why Monte Carlo Tree Search?



The Problem

In games such as Go, the complete game tree is far too large to explore.

MCTS Idea

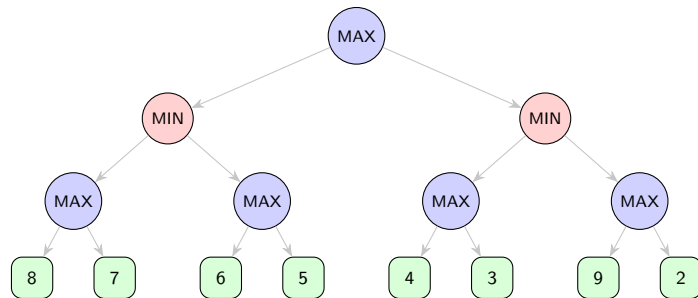
Instead of evaluating every possibility:

- ▶ Sample possible games
- ▶ Record their outcomes
- ▶ Focus on promising moves

Main Idea

MCTS estimates the best move through repeated simulations.

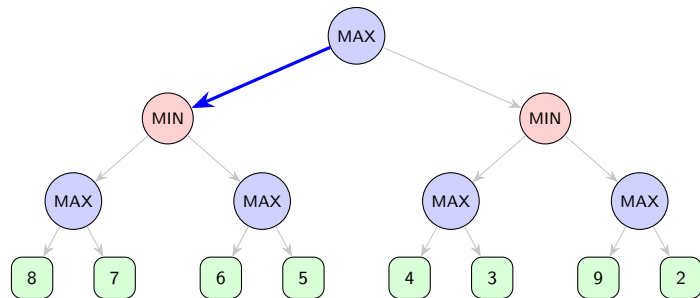
Monte Carlo Tree Search: One Simulation



Start

Begin at the current game state.

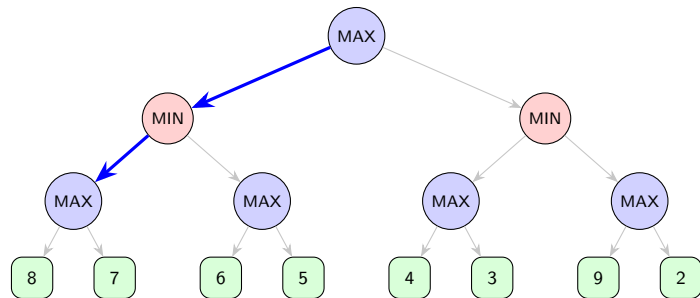
Monte Carlo Tree Search: One Simulation



1. Selection

Choose one available move.

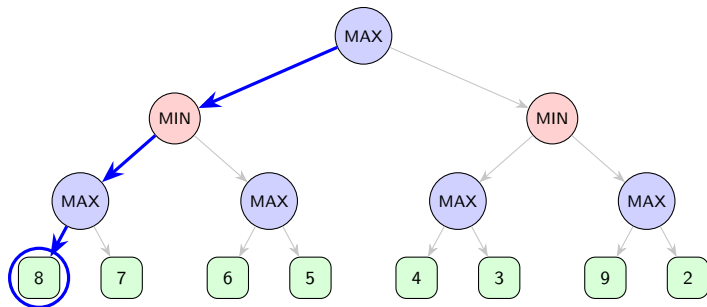
Monte Carlo Tree Search: One Simulation



2. Expansion

Move deeper into the game tree.

Monte Carlo Tree Search: One Simulation

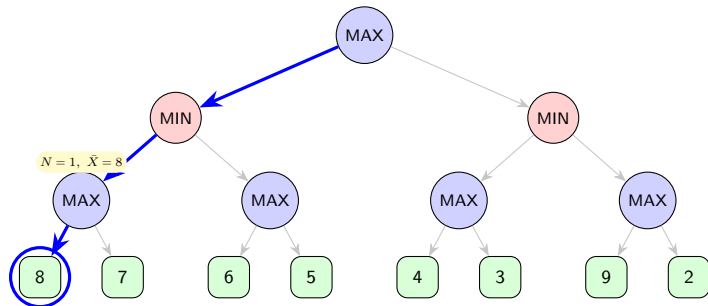


3. Simulation

Complete one possible game.

Outcome = 8

Monte Carlo Tree Search: One Simulation



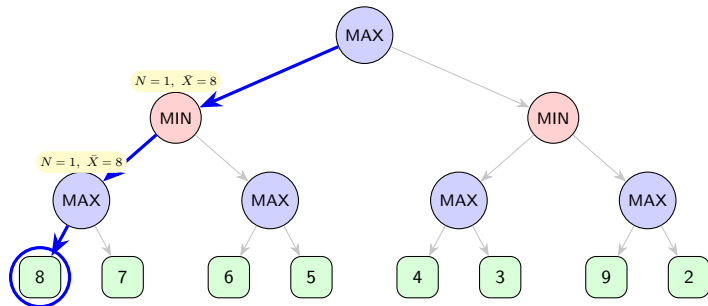
4. Backpropagation

Send the result back through every visited node.

N = number of visits

$\bar{X}$ = average outcome

Monte Carlo Tree Search: One Simulation



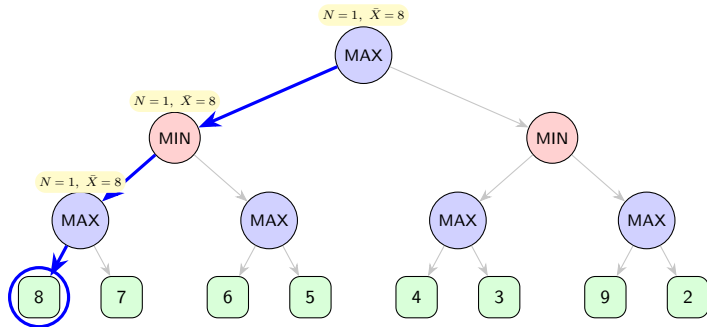
4. Backpropagation

Send the result back through every visited node.

N = number of visits

$\bar{X}$ = average outcome

Monte Carlo Tree Search: One Simulation



4. Backpropagation

Send the result back through every visited node.

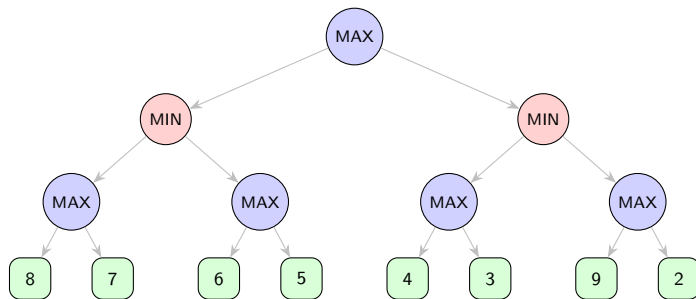
N = number of visits

$\bar{X}$ = average outcome

Important

One simulation gives evidence, not a guaranteed answer.

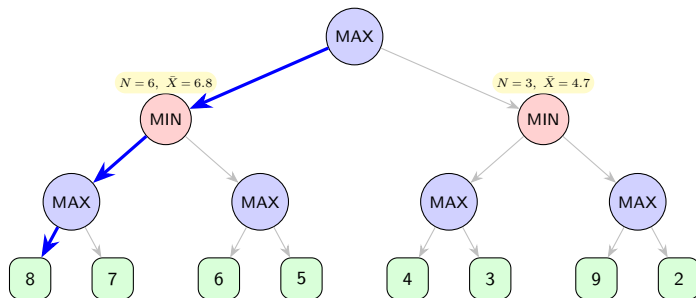
Monte Carlo Tree Search: Repeated Simulations



Repeat

MCTS performs the same four stages many times.

Monte Carlo Tree Search: Repeated Simulations



After 9 Simulations

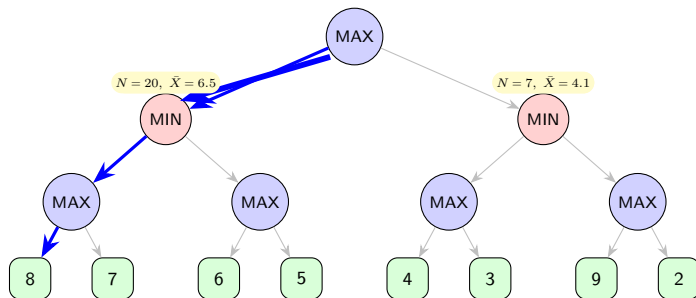
The left branch has:

$$N = 6$$

The right branch has:

$$N = 3$$

Monte Carlo Tree Search: Repeated Simulations

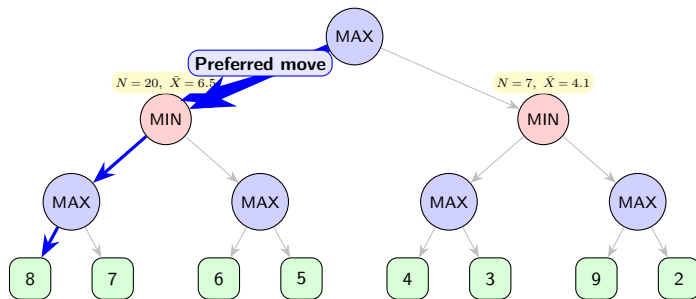


Search Focus

Promising branches receive more simulations.

Uncertain branches are still occasionally explored.

Monte Carlo Tree Search: Repeated Simulations



Decision

MCTS normally chooses the move with the strongest search evidence, often the most visited child.

Exploration vs. Exploitation

Explore uncertain moves, but spend more time on promising moves.

Minimax, Alpha-Beta, and MCTS

Property	Minimax	Alpha-Beta	MCTS
Main strategy	Complete search	Search with pruning	Repeated simulations
Evaluates every branch	Yes	Not always	No
Uses random sampling	No	No	Yes
Result	Exact	Exact	Estimated
Can stop early	Usually no	Usually no	Yes
Best suited for	Small game trees	Searchable game trees	Extremely large trees
Typical example	Simple games	Chess engines	Go and AlphaGo

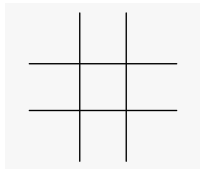
Main Takeaway

Minimax tries to **solve the tree**. MCTS tries to **estimate the best move through experience**.

Selection → Expansion → Simulation → Backpropagation

From Theory to Practice

Minimax



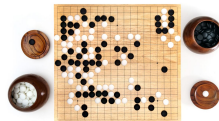
Perfect-information games with small search spaces.

Alpha-Beta Pruning



Modern chess engines (e.g. Deep Blue).

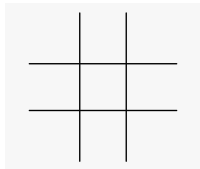
Monte Carlo Tree Search



Large search spaces (e.g. Go).

From Theory to Practice

Minimax



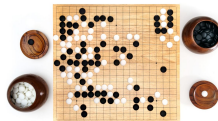
Perfect-information games with small search spaces.

Alpha-Beta Pruning



Modern chess engines (e.g. Deep Blue).

Monte Carlo Tree Search



Large search spaces (e.g. Go).

AlphaGo vs. Lee Sedol (DeepMind, 2016)



"I thought AlphaGo was based on probability calculation and that it was merely a machine. But when I saw this move, I changed my mind. Surely, AlphaGo is creative."

<https://deepmind.google/research/alphago/>

Session Summary

Today you learned:

- ▶ Blind Search
 - ▶ Breadth-First Search (BFS)
 - ▶ Depth-First Search (DFS)
- ▶ Heuristic Search
 - ▶ Heuristics
 - ▶ A*
- ▶ Adversarial Search
 - ▶ Minimax
 - ▶ Alpha-Beta Pruning
 - ▶ Monte Carlo Tree Search

Session Summary

Today you learned:

- ▶ Blind Search
 - ▶ Breadth-First Search (BFS)
 - ▶ Depth-First Search (DFS)
- ▶ Heuristic Search
 - ▶ Heuristics
 - ▶ A*
- ▶ Adversarial Search
 - ▶ Minimax
 - ▶ Alpha-Beta Pruning
 - ▶ Monte Carlo Tree Search

Coming Next: Learning from Experience

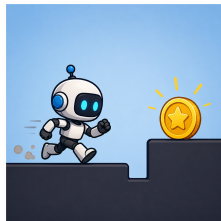
Reinforcement Learning

Instead of searching through possible decisions,
Can an AI improve without being
explicitly programmed?

Reinforcement Learning in a Nutshell

Today: AI searched for the best decision.

Next time: AI will learn how to make better decisions.



Pseudo-code: A* Search

A* Search

Function AStar(start, goal)

Open $\leftarrow$ { start }

Closed $\leftarrow \emptyset$

While Open is not empty

node $\leftarrow$ state in Open with lowest $f(n)$

If node is the goal

return (solution path, total cost)

Remove node from Open

Add node to Closed

For each successor of node

 Compute $g(n)$, $h(n)$ and $f(n)$

If successor is new or has a better path

 Update its cost and parent

 Add successor to Open

return Failure

Pseudo-code: Minimax

Minimax

```
Function Minimax(node)

If node is terminal
    return (None, utility(node))

If node belongs to MAX
    bestMove ← None
    bestValue ←  $-\infty$ 

    For each legal move from node
        child ← result(node, move)
        (ignoredMove, value) ← Minimax(child)

        If value > bestValue
            bestValue ← value
            bestMove ← move

Else // node belongs to MIN
    bestMove ← None
    bestValue ←  $+\infty$ 

    For each legal move from node
        child ← result(node, move)
        (ignoredMove, value) ← Minimax(child)

        If value < bestValue
            bestValue ← value
            bestMove ← move

return (bestMove, bestValue)
```

Pseudo-code: Alpha-Beta Pruning

Alpha-Beta

```
Function AlphaBeta(node,  $\alpha$ ,  $\beta$ )

If node is terminal
    return (None, utility(node))

If node belongs to MAX
    bestMove  $\leftarrow$  None
    bestValue  $\leftarrow -\infty$ 

    For each legal move from node
        child  $\leftarrow$  result(node, move)
        (ignoredMove, value)  $\leftarrow$  AlphaBeta(child,  $\alpha$ ,  $\beta$ )

        If value > bestValue
            bestValue  $\leftarrow$  value
            bestMove  $\leftarrow$  move

     $\alpha \leftarrow \max(\alpha, \text{bestValue})$ 
    If  $\alpha \geq \beta$ 
        break // prune remaining moves

Else // node belongs to MIN
    bestMove  $\leftarrow$  None
    bestValue  $\leftarrow +\infty$ 

    For each legal move from node
        child  $\leftarrow$  result(node, move)
        (ignoredMove, value)  $\leftarrow$  AlphaBeta(child,  $\alpha$ ,  $\beta$ )

        If value < bestValue
            bestValue  $\leftarrow$  value
            bestMove  $\leftarrow$  move

     $\beta \leftarrow \min(\beta, \text{bestValue})$ 
    If  $\alpha \geq \beta$ 
        break // prune remaining moves

return (bestMove, bestValue)
```

Calling Alpha-Beta

$(\text{bestMove}, \text{bestValue}) = \text{AlphaBeta}(\text{root}, -\infty, +\infty)$

Pseudo-code: Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS)

```
Function MCTS(root, budget)

Initialize visit counts and rewards

While computation budget remains

    node ← Select the most promising node
           // high average reward + exploration

    If node is not terminal
        node ← Expand(node)

    reward ← Simulate(node)

    Update visit counts and average rewards
           // Backpropagation

bestChild ← root child with highest visit count

bestMove ← move leading to bestChild
bestValue ← average reward of bestChild

return (bestMove, bestValue)
```

Key Idea

Each simulation updates the statistics, which guide future simulations toward more promising parts of the search tree.